

BÁO CÁO KỸ THUẬT ĐỒ ÁN CUỐI KỲ

BlockFL

Học liên kết bảo mật dữ liệu cảm biến y tế / đô thị tích hợp Blockchain

Học phần: Cơ sở Blockchain và Ứng dụng

GVHD: Huỳnh Thế Thiện

Nhóm thực hiện: [Điền tên nhóm]

Mã nguồn: [Điền link GitHub repository]

STT	Họ tên – MSSV	Vai trò
1	[Điền họ tên – MSSV]	Blockchain Developer
2	[Điền họ tên – MSSV]	AI/ML Engineer
3	[Điền họ tên – MSSV]	IoT/Backend Integrator
4	[Điền họ tên – MSSV]	Dashboard/Frontend Developer
5	[Điền họ tên – MSSV]	PM & Báo cáo/Thuyết trình

Tháng 9/2026

Mục lục

Mục lục	2
1. Giới thiệu	3
1.1. Bối cảnh và bài toán.....	3
1.2. Mục tiêu đồ án	3
2. Kiến trúc hệ thống tổng quan.....	3
3. Thiết kế chi tiết từng tầng	4
3.1. Tầng IoT (Edge Node)	4
3.2. Tầng AI (Federated Learning)	4
3.2.1. Kiến trúc mô hình	4
3.2.2. Huấn luyện cục bộ (local training).....	4
3.2.3. Tổng hợp FedAvg	5
3.2.4. Vòng lặp huấn luyện và tiêu chí đánh giá	5
3.3. Tầng Blockchain (Smart Contract)	5
3.3.1. BlockFLToken.sol — token ERC-20 thưởng	6
3.3.2. FederatedAggregator.sol — điều phối round FL	6
4. Công nghệ sử dụng	6
5. Kết quả thực nghiệm.....	6
5.1. Hội tụ của Global Model qua từng round	6
5.2. So sánh FedAvg với từng node tự huấn luyện riêng lẻ	7
5.3. Kiểm định độ ổn định qua nhiều lần chia dữ liệu	8
5.4. Kiểm thử smart contract.....	9
5.5. Minh bạch và chống chối bỏ trên chuỗi	9
6. Đánh giá rủi ro và phương án xử lý	9
7. Kết luận và hướng phát triển.....	10
8. Cấu trúc repository GitHub.....	10
9. Phân công nhiệm vụ.....	10
10. Tài liệu tham khảo	11

1. Giới thiệu

1.1. Bối cảnh và bài toán

Trong các hệ thống IoT y tế/đô thị (thiết bị đeo theo dõi nhịp tim, cảm biến chất lượng không khí...), dữ liệu thô thường rất nhạy cảm và việc gửi toàn bộ về server trung tâm để huấn luyện AI tiềm ẩn rủi ro rò rỉ thông tin, đồng thời tạo điểm nghẽn (single point of failure).

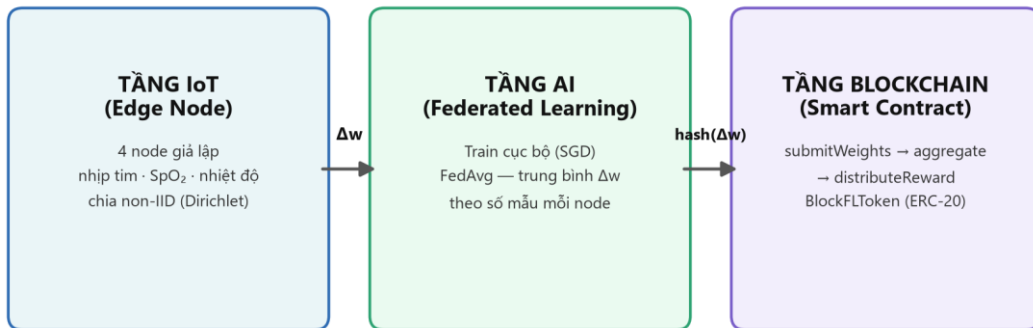
Federated Learning (FL) giải quyết vấn đề riêng tư bằng cách huấn luyện cục bộ tại từng node biên, chỉ gửi trọng số mô hình (Δw) thay vì dữ liệu thô. Tuy nhiên, FL truyền thống vẫn cần một server tổng hợp (aggregator) đáng tin cậy — đây là điểm mà Blockchain thay thế: minh bạch hoá quá trình tổng hợp trọng số, chống gian lận, và tự động thưởng token cho các node đóng góp trung thực.

1.2. Mục tiêu đề án

Xây dựng hệ thống CPS mô phỏng end-to-end: IoT Edge Node (giả lập cảm biến) → Federated Learning (huấn luyện cục bộ, FedAvg) → Smart Contract (tổng hợp trọng số on-chain qua hash, phân phối token thưởng), kèm dashboard giám sát thời gian thực và bộ thực nghiệm kiểm chứng độ ổn định của mô hình.

2. Kiến trúc hệ thống tổng quan

Hệ thống gồm 3 tầng ghép nối tuần tự, đúng mô hình Cyber-Physical System (CPS) mà đề cương yêu cầu:



Dữ liệu thô KHÔNG rời khỏi node — chỉ trọng số (Δw) và mã băm 32 byte đi lên trên

Hình 1. Kiến trúc 3 tầng CPS: IoT → AI (Federated Learning) → Blockchain

Luồng dữ liệu của một round huấn luyện diễn ra như sau:

- Mỗi edge node (giả lập) đọc một lô dữ liệu cảm biến (nhịp tim, SpO₂, nhiệt độ cơ thể).
- Node huấn luyện cục bộ vài epoch trên dữ liệu của mình → sinh ra bộ trọng số Δw .
- Node băm Δw (keccak256) và gọi submitWeights(hash, numSamples, round) lên smart contract.
- Khi đủ số node báo cáo, quy trình FedAvg (tính off-chain) tổng hợp Δw theo trọng số là số mẫu mỗi node, sinh ra Global Model mới; hash của model mới được ghi on-chain qua aggregate().

- Hợp đồng tự động phân bổ token thưởng (distributeReward()) theo tỉ lệ số mẫu đóng góp.
- Các node tải Global Model mới về, lặp lại round tiếp theo.

3. Thiết kế chi tiết từng tầng

3.1. Tầng IoT (Edge Node)

4 edge node giả lập, mỗi node đại diện một bệnh viện/trạm quan trắc khác nhau. Dữ liệu (nhịp tim, SpO₂, nhiệt độ cơ thể, biến thiên nhịp tim, chuyển động) được sinh ngẫu nhiên theo phân phối Gauss cho 2 lớp bình thường/bất thường, sau đó chia cho từng node theo phân phối Dirichlet ($\alpha = 0.4$) để tạo tính non-IID — tỉ lệ ca bất thường lệch hẳn nhau giữa các node, đúng thực tế lâm sàng (bệnh viện chuyên khoa cấp cứu sẽ có tỉ lệ ca nặng cao hơn phòng khám định kỳ).

Node	Số mẫu train	Số mẫu test	Bình thường	Bất thường	Tỉ lệ bất thường
node 0	226	75	64	162	71.7%
node 1	368	122	297	71	19.3%
node 2	315	105	212	103	32.7%
node 3	290	97	169	121	41.7%

Bảng 1. Phân bố dữ liệu non-IID giữa 4 edge node (nguồn: data/nodes_preview.json)

3.2. Tầng AI (Federated Learning)

3.2.1. Kiến trúc mô hình

Bài toán đặt ra là phân loại nhị phân (bình thường / bất thường) trên 5 đặc trưng số dạng bảng (heart_rate, spo2, body_temp, hr_var, motion) — không phải ảnh hay chuỗi thời gian dài, nên không cần đến CNN hay RNN/LSTM vốn nặng và khó hội tụ nhanh trong phạm vi vài round FedAvg. Nhóm chọn mạng nơ-ron truyền thẳng (MLP — VitalsMLP trong ai_model/model.py) gồm:

- Lớp vào: 5 nơ-ron, tương ứng 5 đặc trưng cảm biến đã chuẩn hoá (z-score theo thống kê của tập train từng node).
- 2 lớp ẩn, mỗi lớp 16 nơ-ron, kích hoạt ReLU — đủ biểu diễn ranh giới phi tuyến giữa 2 lớp mà không dư thừa tham số (tổng cộng chỉ ~450 tham số/mô hình).
- Lớp ra: 2 nơ-ron (logit cho 2 lớp), kết hợp hàm mất mát CrossEntropyLoss (tương đương softmax + negative log-likelihood).

Chọn mô hình nhỏ là chủ đích: (1) FedAvg cần vài chục round là hội tụ trong giới hạn thời gian đồ án; (2) tham số càng ít thì Δw truyền giữa node và server (off-chain) càng nhẹ, phù hợp mô phỏng cho thiết bị biên tài nguyên hạn chế; (3) dễ debug và trực quan hoá kết quả khi thuyết trình.

3.2.2. Huấn luyện cục bộ (local training)

Ở mỗi round, mỗi node nhận global weights, khởi tạo lại mô hình với đúng bộ trọng số đó (không huấn luyện lại từ đầu), rồi huấn luyện tiếp trên dữ liệu riêng của mình với cấu hình:

Siêu tham số	Giá trị	Vai trò
Optimizer	SGD, momentum = 0.9	Momentum giúp hội tụ nhanh hơn SGD thuần trên tập dữ liệu nhỏ mỗi node
Learning rate	0.05	Đủ lớn để hội tụ trong 5 epoch, nhưng không quá lớn gây dao động Δw giữa các node

Siêu tham số	Giá trị	Vai trò
Epoch cục bộ	5	Cân bằng giữa hội tụ cục bộ tốt và tránh overfit lên dữ liệu non-IID của riêng node
Batch size	32	Phù hợp với quy mô dữ liệu mỗi node (153–290 mẫu train)
Hàm mất mát	CrossEntropyLoss	Chuẩn cho bài toán phân loại đa lớp (ở đây là 2 lớp)

Sau khi huấn luyện xong, node chỉ gửi đi bộ trọng số mới (Δw dưới dạng vector phẳng, ghép từ toàn bộ tham số của mô hình) chứ không gửi gradient hay dữ liệu — đây chính là ranh giới bảo mật cốt lõi của Federated Learning.

3.2.3. Tổng hợp FedAvg

Thuật toán tổng hợp là FedAvg (Federated Averaging – McMahan et al., 2017), ý tưởng cốt lõi là lấy trung bình có trọng số các bộ tham số cục bộ, trọng số tỉ lệ với số mẫu mỗi node đóng góp:

$$w_{\text{global}} = \sum (n_i / N) \times w_i, \text{ với } n_i \text{ là số mẫu của node } i, N = \sum n_i$$

Cách chia trọng số theo n_i (thay vì chia đều $1/\text{số node}$) đảm bảo công bằng thống kê: node có nhiều dữ liệu hơn (ví dụ node 1 với 368 mẫu) sẽ có ảnh hưởng lớn hơn tới mô hình chung so với node ít dữ liệu (node 0 với 226 mẫu) — đúng tinh thần "đóng góp nhiều, ảnh hưởng nhiều", đồng thời cũng là căn cứ để tăng Blockchain tính token thưởng theo đúng công thức tương tự (mục 3.3.2).

Việc tính trung bình được thực hiện off-chain (trong `ai_model/fedavg.py`) vì: chi phí gas để nhân/cộng hàng trăm số thực dấu phẩy động ngay trên EVM là không cần thiết và tốn kém; EVM cũng không có kiểu dữ liệu số thực gốc. Sau khi tính xong, toàn bộ vector trọng số mới được băm bằng keccak256 thành 32 byte duy nhất, và chỉ giá trị hash này được ghi lên smart contract qua hàm `aggregate()` — chuỗi đóng vai trò trọng tài xác nhận "phiên bản nào là bản chính thức", không đóng vai trò máy tính.

3.2.4. Vòng lặp huấn luyện và tiêu chí đánh giá

- Bước 1 — Khởi tạo: Global Model khởi tạo ngẫu nhiên (round 0), dùng làm baseline so sánh.
- Bước 2 — Broadcast: global weights hiện tại được gửi (giả lập, gọi hàm trực tiếp trong `run_demo.py`) xuống toàn bộ 4 node.
- Bước 3 — Local training: mỗi node huấn luyện độc lập theo cấu hình ở mục 3.2.2.
- Bước 4 — Submit: node băm Δw và gọi `submitWeights()` lên smart contract kèm số mẫu n_i .
- Bước 5 — Aggregate: khi đủ 4/4 node đã nộp (`minNodes = 3`, có thể chốt sớm hơn), chạy FedAvg off-chain, ghi hash lên chain.
- Bước 6 — Đánh giá: Global Model mới được chấm lại trên tập test gộp của toàn bộ 4 node (chưa từng thấy trong huấn luyện) để đo accuracy, F1 (lớp bất thường) và loss — đây chính là 3 chỉ số dùng để vẽ đường hội tụ ở Hình 2.
- Bước 7 — Thưởng & lặp: hợp đồng phân phối token BFL theo tỉ lệ n_i , sau đó mở round tiếp theo (`advanceRound()`), quay lại Bước 2.

Việc luôn đánh giá trên tập test gộp (không phải tập train) ở mọi round là điều kiện bắt buộc để đường cong ở Hình 2 phản ánh đúng khả năng tổng quát hoá của mô hình, chứ không phải hiện tượng học thuộc lòng dữ liệu cục bộ. Đây cũng là cơ sở cho phép so sánh trực tiếp accuracy của Global Model với accuracy của từng node tự huấn luyện riêng lẻ ở mục 5.2 — vì cả hai đều được chấm trên chính xác cùng một tập test.

3.3. Tầng Blockchain (Smart Contract)

Ngôn ngữ Solidity 0.8.24, phát triển và kiểm thử trên Hardhat local network. Hai smart contract:

3.3.1. BlockFLToken.sol — token ERC-20 thưởng

Token BFL tối giản, tuân thủ chuẩn ERC-20 (transfer, approve, transferFrom). Chỉ địa chỉ minter (được set là FederatedAggregator sau khi deploy) mới được phép mint — chống việc chủ hợp đồng tự phát token cho mình.

3.3.2. FederatedAggregator.sol — điều phối round FL

Hàm	Người gọi	Chức năng
registerNode(address)	owner	Đăng ký node tham gia mạng
submitWeights(hash, n, round)	node đã đăng ký	Nộp hash Δw + số mẫu cho round hiện tại
aggregate(round, globalHash)	owner	Chốt hash global model khi đủ minNodes đã nộp
distributeReward(round)	owner	Mint token BFL theo tỉ lệ $\text{numSamples}/\text{totalSamples}$
advanceRound()	owner	Mở round tiếp theo sau khi round hiện tại đã chốt

Bảng 2. Các hàm chính của FederatedAggregator.sol

Ba sự kiện WeightsSubmitted, ModelAggregated, RewardDistributed được phát ra ở mỗi bước tương ứng, tạo thành nhật ký minh bạch, bất biến — bất kỳ ai cũng đọc lại được lịch sử đóng góp mà không cần tin vào một bên trung gian. Trọng số Δw thật (vài KB đến vài trăm KB) được lưu file .npy off-chain; on-chain chỉ giữ 32 byte hash keccak256, giảm chi phí gas so với lưu toàn bộ tensor.

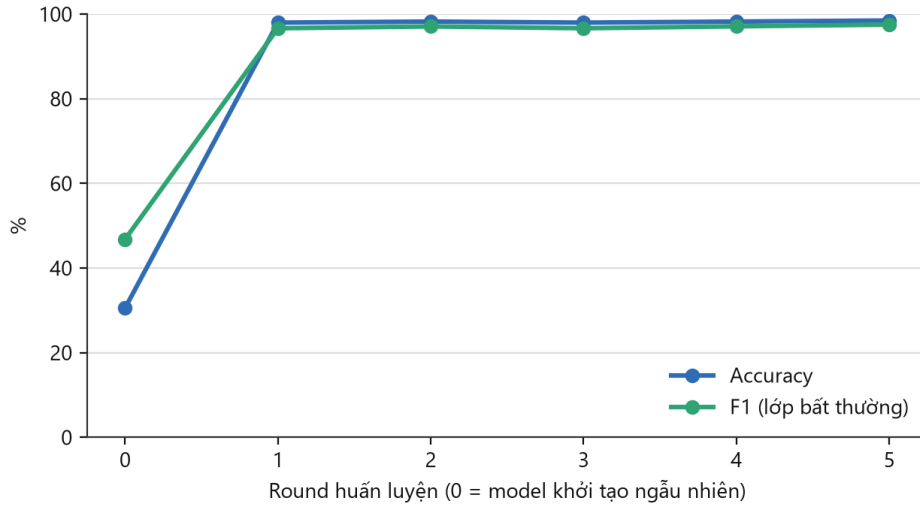
4. Công nghệ sử dụng

Thành phần	Công nghệ
Ngôn ngữ AI	Python 3.14, PyTorch
Ngôn ngữ Smart Contract	Solidity 0.8.24
Framework blockchain	Hardhat (dev/test, EVM local)
Kết nối Web3	Web3.py (backend AI ↔ chain), Ethers.js (dashboard)
Lưu trữ trọng số	File .npy off-chain + hash keccak256 on-chain
Dashboard	HTML/CSS/JS thuần, 8 trang, cập nhật realtime mỗi 5s
Version control	Git/GitHub

5. Kết quả thực nghiệm

5.1. Hội tụ của Global Model qua từng round

Chạy 5 round trên 4 node (seed dữ liệu = 4, tổng 1199 mẫu). Kết quả: accuracy tăng từ 30.58% (model khởi tạo ngẫu nhiên) lên 98.50%, F1 lớp bất thường từ 46.83% lên 97.52%, loss giảm từ 0.754 xuống 0.048 — toàn bộ mức tăng này đến từ FedAvg, không node nào gửi dữ liệu thô đi đâu cả.



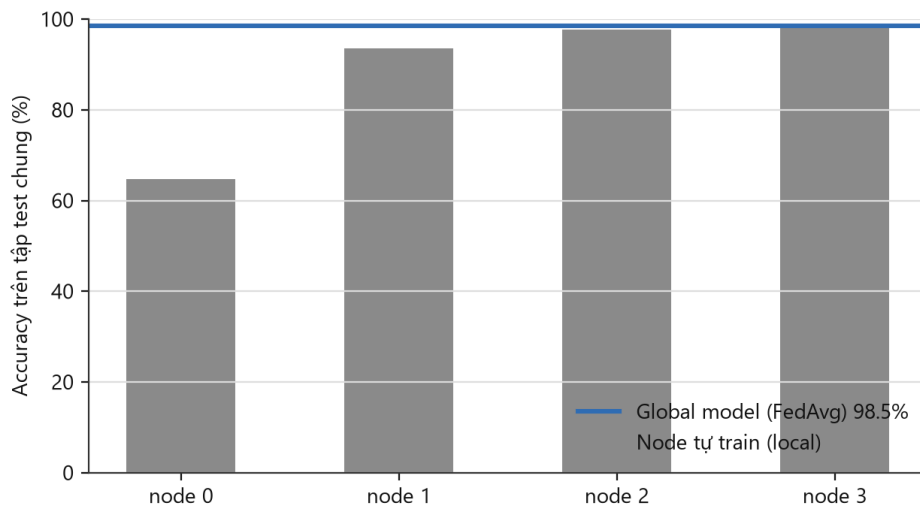
Hình 2. Độ hội tụ accuracy/F1 qua các round (nguồn: results.json)

Round	Accuracy	F1	Loss	Global model hash
0 (khởi tạo)	30.58%	46.83%	0.7535	—
1	97.99%	96.67%	0.0777	c6024e6d5444f8...
2	98.25%	97.10%	0.0609	95ba6d01babaed...
3	97.99%	96.67%	0.0559	d97fffb8cf1324...
4	98.25%	97.12%	0.0512	178c6453025dbf...
5	98.50%	97.52%	0.0479	8e355a5eb8fb43...

Bảng 3. Chi tiết từng round, hash đọc trực tiếp từ smart contract

5.2. So sánh FedAvg với từng node tự huấn luyện riêng lẻ

Để chứng minh giá trị của Federated Learning (không chỉ là chạy cho có), mỗi local model (chỉ train trên dữ liệu của 1 node) được chấm trên cùng tập test chung với global model. Ở round cuối:



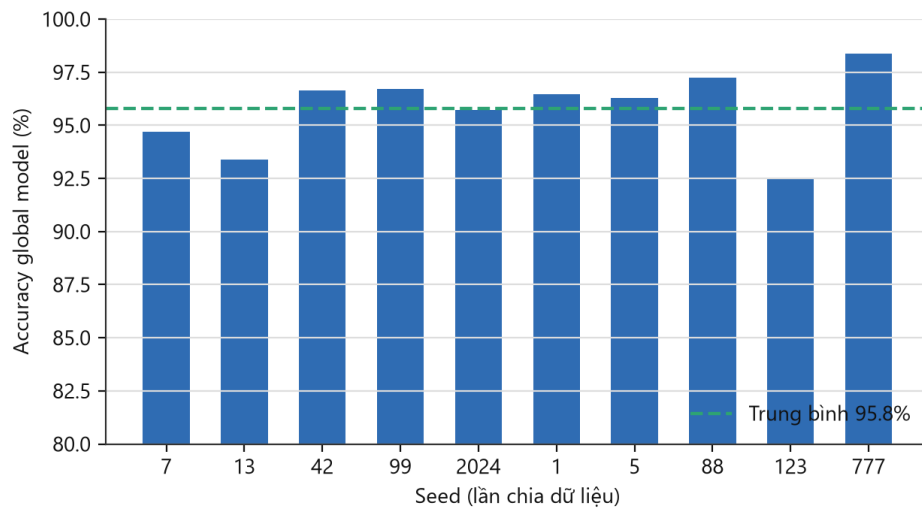
Hình 3. Global model (FedAvg) so với từng node tự train một mình

Node	Số mẫu	Accuracy tự train (trên tập test chung)	Accuracy global model trên dữ liệu riêng của node
node 0	226	64.66%	90.27%
node 1	368	93.48%	97.55%
node 2	315	97.74%	100.00%
node 3	290	98.25%	100.00%

Bảng 4. Global model đạt 98.50%, vượt trội mọi node tự train — node dữ liệu lệch nặng nhất (ít mẫu, tỉ lệ bất thường cao) tự train chỉ đạt độ chính xác thấp hơn hẳn.

5.3. Kiểm định độ ổn định qua nhiều lần chia dữ liệu

Một kết quả duy nhất trên một lần chia dữ liệu có thể chỉ là seed may mắn. Toàn bộ pipeline được chạy lại trên 10 seed khác nhau (mỗi seed = một bộ dữ liệu cảm biến hoàn toàn khác — tỉ lệ bất thường, số mẫu, giá trị cảm biến của mọi node đều đổi):



Hình 4. Accuracy của global model qua 10 lần chia dữ liệu khác nhau

Chỉ số	Giá trị
Accuracy trung bình	95.81% ± 1.8 điểm %
Accuracy thấp nhất / cao nhất	92.47% / 98.37%
F1 lớp bất thường trung bình	94.16% ± 2.4 điểm %
FedAvg thắng mọi node tự train	10/10 lần
Cách biệt trung bình so với node tốt nhất	+1.4 điểm %

Bảng 5. Tóm tắt độ ổn định (nguồn: results_sweep.json, sinh bởi scripts/run_sweep.py)

Seed	Global (FedAvg)	Node tốt nhất tự train	Node kém nhất tự train	Cách biệt	Kết quả
7	94.70%	94.35%	62.19%	+0.4 %	Thắng
13	93.40%	91.75%	59.08%	+1.7 %	Thắng
42	96.66%	94.22%	71.43%	+2.4 %	Thắng
99	96.73%	96.43%	86.01%	+0.3 %	Thắng
2024	95.73%	94.59%	69.23%	+1.1 %	Thắng
1	96.47%	92.16%	49.02%	+4.3 %	Thắng
5	96.31%	96.02%	45.45%	+0.3 %	Thắng
88	97.24%	96.13%	64.64%	+1.1 %	Thắng
123	92.47%	90.65%	66.23%	+1.8 %	Thắng
777	98.37%	98.04%	72.55%	+0.3 %	Thắng

Bảng 6. Chi tiết 10 lần chạy độc lập — FedAvg thắng mọi node tự train ở cả 10/10 lần

5.4. Kiểm thử smart contract

Bộ test Hardhat (test/aggregator.test.js) kiểm tra 6 kịch bản: đăng ký node và chặn đăng ký trùng, chặn node lạ gọi submitWeights, chặn nộp sai round/nộp 2 lần, chặn aggregate khi chưa đủ minNodes, luồng đầy đủ submit → aggregate → distributeReward → advanceRound (đối chiếu số token đúng công thức chia theo tỉ lệ mẫu), và chặn tài khoản không phải owner gọi các hàm quản trị. Toàn bộ 6/6 test pass.

5.5. Minh bạch và chống chối bỏ trên chuỗi

Dashboard (trang Round on-chain) tự động bấm lại file trọng số cục bộ ai_model/weights/global_round_N.npy và so với globalModelHash đang lưu trên smart contract. Nếu khớp — hiển thị badge “Khớp” — nghĩa là file trọng số đang giữ đúng là file đã được chốt on-chain; chỉ cần sửa 1 byte trong file là badge lập tức chuyển sang “Lệch”, trong khi giá trị giả trên chuỗi thì không ai sửa được để che giấu, thể hiện đúng tính chất bất biến của blockchain.

6. Đánh giá rủi ro và phương án xử lý

Rủi ro	Phương án xử lý
Gas fee/độ trễ khi test nhiều lần trên Testnet	Phát triển và test chủ yếu trên Hardhat local; chỉ deploy Sepolia ở giai đoạn cuối để demo
Trọng số mô hình quá lớn để lưu on-chain	Lưu hash keccak256 (32 byte) on-chain, trọng số thật lưu off-chain (file .npy)
Mô hình FL không hội tụ do dữ liệu non-IID quá lệch	Điều chỉnh mức Dirichlet α , tăng số round, dùng learning rate nhỏ; đã kiểm chứng hội tụ ổn định qua 10 seed khác nhau (mục 5.3)
Node gian lận gửi Δw bất thường (poisoning)	Hướng phát triển: kiểm tra độ lệch Δw trước khi tổng hợp (chưa triển khai trong bản hiện tại)
Một kết quả FL có thể chỉ là “ăn may” theo 1 lần chia dữ liệu	Chạy lại toàn bộ pipeline trên 10 seed độc lập (scripts/run_sweep.py) — FedAvg thắng mọi node tự train ở cả 10/10 lần

7. Kết luận và hướng phát triển

Hệ thống BlockFL đã hiện thực hoá thành công luồng end-to-end IoT → Federated Learning → Blockchain: 4 edge node với dữ liệu non-IID huấn luyện cục bộ, FedAvg tổng hợp trọng số off-chain, smart contract ghi nhận minh bạch qua hash và tự động phân phối token thưởng theo đóng góp. Kết quả thực nghiệm cho thấy mô hình hội tụ ổn định (accuracy trung bình 95.8% qua 10 lần chia dữ liệu độc lập) và FedAvg luôn vượt trội so với việc từng node tự huấn luyện riêng lẻ.

Hướng phát triển tiếp theo:

- Cơ chế phát hiện node gian lận: kiểm tra độ lệch thống kê của Δw trước khi đưa vào aggregate().
- Deploy chính thức lên Testnet Sepolia kèm liên kết Etherscan cho từng giao dịch.
- Nhúng mô hình rút gọn (TensorFlow Lite) lên phần cứng thật (ESP32/Raspberry Pi) thay cho giả lập hoàn toàn bằng phần mềm.
- Lưu trữ trọng số trên IPFS thay vì file cục bộ để tăng tính phi tập trung.

8. Cấu trúc repository GitHub

Mã nguồn được tổ chức theo đúng cấu trúc thư mục quy định trong tài liệu hướng dẫn của học phần:

- README.md — hướng dẫn cài đặt, cấu hình môi trường và các bước chạy demo.
- contracts/ — BlockFLToken.sol, FederatedAggregator.sol.
- test/ — aggregator.test.js (6 test Hardhat).
- scripts/ — deploy.js (deploy contract), web3_interface.py (cầu nối Web3.py), run_sweep.py (kiểm định đa seed), build_report.js (sinh báo cáo này).
- ai_model/ — model.py, local_train.py, fedavg.py và thư mục weights/ lưu trọng số theo round.
- iot_code/ — edge_node.py, data_partition.py (giả lập & chia dữ liệu non-IID).
- dashboard/ — 8 trang giám sát thời gian thực (HTML/CSS/JS + Ethers.js).
- run_demo.py — script điều phối chạy toàn bộ pipeline end-to-end.

Repository công khai (Public) trên GitHub, link: [Điền link GitHub repository].

9. Phân công nhiệm vụ

Đề cương ban đầu phân công 5 vai trò như sau (điền cụ thể tên/MSSV từng thành viên trước khi nộp):

Vai trò	Nhiệm vụ chính
Blockchain Developer	Viết/test/deploy smart contract, tích hợp token ERC-20, quản lý Testnet
AI/ML Engineer	Xây dựng mô hình, cài đặt FedAvg, chia dữ liệu non-IID, đánh giá hội tụ
IoT/Backend Integrator	Giả lập edge node, viết cầu nối Web3.py giữa AI và blockchain
Dashboard/Frontend Developer	Xây dựng dashboard Web3, tích hợp MetaMask, hiển thị realtime
PM & Báo cáo/Thuyết trình	Quản lý tiến độ, tổng hợp báo cáo kỹ thuật, chuẩn bị demo

10. Tài liệu tham khảo

- McMahan, H. B., et al. (2017). Communication-Efficient Learning of Deep Networks from Decentralized Data.
- Tài liệu Solidity chính thức: <https://docs.soliditylang.org>
- Hardhat Documentation: <https://hardhat.org>
- Web3.py Documentation: <https://web3py.readthedocs.io>
- Ethers.js Documentation: <https://docs.ethers.org>